Section 4.2: The Convolution Operation

Kernels, Padding, Stride, Channels and Gradients

AI-IM Class — Lecture Notes

In Section 4.1 we slid a small kernel along a signal and saw why this beats a fully connected layer for images. In this section we make the operation precise and compute it by hand. We

- define convolution in one and two dimensions,
- control the output size with *padding* and *stride*,
- handle colour images and many filters (*channels*),
- write convolution as a matrix, connecting it back to the MLP, and
- compute the gradient of the loss with respect to the kernel.

Notation. As in Section 3.4, bold capitals are arrays and lower-case letters are their entries. The input is $\mathbf{X}$ with entries $x_{i,j}$; the kernel is $\mathbf{W}$ with entries $w_{u,v}$; the output (pre-activation) is $\mathbf{Z}$ with entries $z_{i,j}$; b is the bias. The kernel has size $k \times k$. Do not confuse the bold kernel $\mathbf{W}$ with the italic image width W.

1 Convolution in One Dimension

Let $\mathbf{x} = (x_1, \dots, x_n)$ be an input of length n and $\mathbf{w} = (w_1, \dots, w_k)$ a kernel of length $k \le n$. Place the kernel at position i, multiply the overlapping entries, and add:

$$z_i = \sum_{u=1}^{k} w_u \, x_{i+u-1} + b, \qquad i = 1, 2, \dots, n-k+1.$$

The output has length $n - k + 1$.

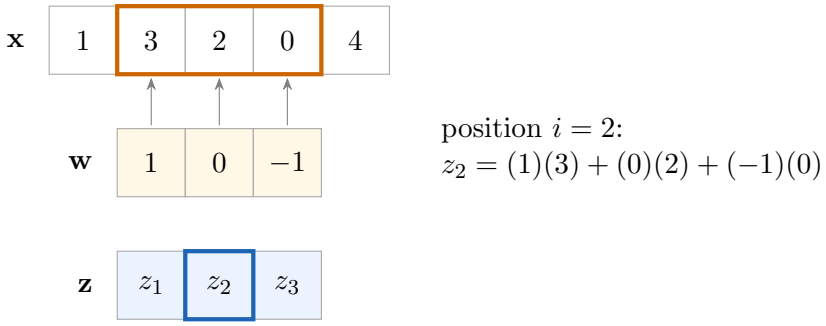


Figure 1: One-dimensional convolution with $n = 5$ and $k = 3$ (bias $b = 0$). The kernel slides one step at a time; the window at position $i = 2$ is outlined.

Exercise 1 A 1-D convolution

Use $\mathbf{x} = (1, 3, 2, 0, 4)$, $\mathbf{w} = (1, 0, -1)$ and $b = 0$, as in Figure 1.

(a) The output has length $5 - 3 + 1 = 3$.

(b) $z_1 = 1 - 2 = -1$, $z_2 = 3 - 0 = 3$, $z_3 = 2 - 4 = -2$.

(c) This kernel computes $z_i = x_i - x_{i+2}$. It measures how much the signal decreases over two steps.

Convolution or cross-correlation? In mathematics and signal processing, *convolution* flips the kernel before sliding it: $(x * w)_i = \sum_u x_{i-u}\, w_u$. The operation in the box above, which does not flip, is strictly called **cross-correlation**. With the flipped kernel $(-1, 0, 1)$, Exercise 1 would give $\mathbf{z} = (1, -3, 2)$: the same numbers with opposite sign.

In a CNN the kernel values are *learned*. If the flipped version is better, training simply learns the flipped numbers, so the choice makes no difference. Deep-learning libraries, including PyTorch's `nn.Conv1d` and `nn.Conv2d`, compute cross-correlation and call it convolution. We do the same.

2 Convolution in Two Dimensions

For an image the kernel is a small $k \times k$ grid that slides across both rows and columns. At each position we multiply the kernel entrywise with the $k \times k$ patch beneath it and add everything up.

$$z_{i,j} = \sum_{u=1}^{k} \sum_{v=1}^{k} w_{u,v}\; x_{i+u-1,\, j+v-1} + b.$$

For an $H \times W$ input the output is $(H - k + 1) \times (W - k + 1)$.

The output is called a **feature map**: entry $z_{i,j}$ says how strongly the pattern in the kernel is present in the patch whose top-left corner is pixel (i, j).

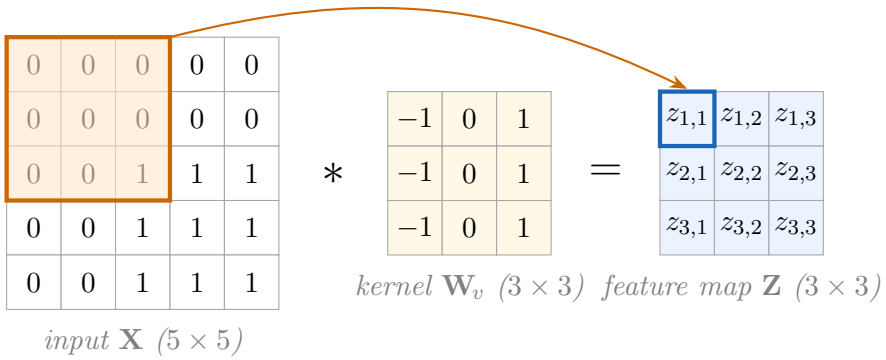


Figure 2: Two-dimensional convolution. The input contains a bright square in its lower-right corner. The outlined 3×3 patch, multiplied entrywise by the kernel and summed, gives $z_{1,1}$. Sliding the patch one column right gives $z_{1,2}$, and so on.

Exercise 2 Detecting edges by hand

Use the input $\mathbf{X}$ of Figure 2 with $b = 0$ and two kernels:

$$\mathbf{W}_v = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}, \qquad \mathbf{W}_h = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}.$$

$\mathbf{W}_v$ computes (right column of the patch) − (left column); $\mathbf{W}_h$ computes (bottom row) − (top row).

$$\mathbf{Z}_v = \begin{pmatrix} 1 & 1 & 0 \\ 2 & 2 & 0 \\ 3 & 3 & 0 \end{pmatrix}, \qquad \mathbf{Z}_h = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 0 & 0 & 0 \end{pmatrix}$$

Where in the image is $\mathbf{Z}_v$ largest, and why?

Bottom-left: those patches contain the whole vertical edge of the square (dark left, bright right in a

The two kernels respond to different features of the *same* image: $\mathbf{W}_v$ to the left (vertical) edge of the square, $\mathbf{W}_h$ to its top (horizontal) edge. Inside the square, where the image is flat, both are zero.

Hand-designed vs. learned kernels. $\mathbf{W}_v$ and $\mathbf{W}_h$ are classical image-processing filters (the *Prewitt* filters), designed by hand. In a CNN the nine numbers in each kernel are *parameters*, found by gradient descent like any other weight. When we look at the kernels a trained CNN learned in its first layer, many turn out to be edge and colour detectors very much like these.

3 Padding

Without help, every convolution shrinks the input: a 3×3 kernel removes one pixel from each border, so a 28×28 image becomes 26×26, then 24×24, and so on. Pixels on the border are also covered by fewer windows than pixels in the middle, so the network sees them less.

The fix is **padding**: surround the input with p rows and columns of zeros before convolving.

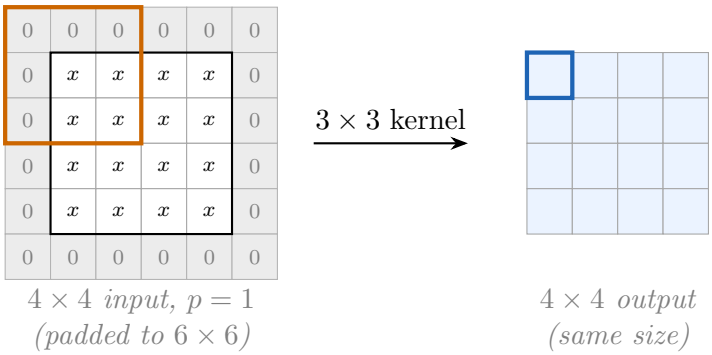


Figure 3: Zero padding with $p = 1$. The first window (orange) is centred on the corner pixel, so the corner now gets its own output, and the output has the same size as the input.

Padding adds $2p$ to each dimension, so the output length becomes $n + 2p - k + 1$. Three choices of p have names.

Type	Padding p	Output length	Use
Valid	0	$n - k + 1$	shrinks; common
Same	$(k-1)/2$ (k odd)	n	keeps size; most common
Full	$k - 1$	$n + k - 1$	grows; rare in CNNs

Same padding is the reason almost all CNN kernels have *odd* size (3×3, 5×5, 7×7): then $(k-1)/2$ is a whole number and the kernel has a centre pixel. In PyTorch, `padding=0` is valid padding, and `padding=1` with a 3×3 kernel (or `padding="same"`) is same padding.

Exercise 3 Same padding in 1-D

Repeat Exercise 1 ($\mathbf{x} = (1, 3, 2, 0, 4)$, $\mathbf{w} = (1, 0, -1)$) with same padding.

(a) $p = 1$; the padded input is $(0,\ 1,\ 3,\ 2,\ 0,\ 4,\ 0)$.

(b) $\mathbf{z} = \big(-3,\ -1,\ 3,\ -2,\ 0\big)$, of length 5.

(c) Compare with Exercise 1: the middle three outputs are the same values $(-1, 3, -2)$; padding only adds outputs at the borders.

4 Stride

So far the kernel has moved one step at a time. With **stride** s it moves s steps, so only every s-th output is computed. A stride of 2 roughly halves the output in each dimension. This reduces computation and, as noted in Section 4.1, makes the receptive field of later layers grow faster.

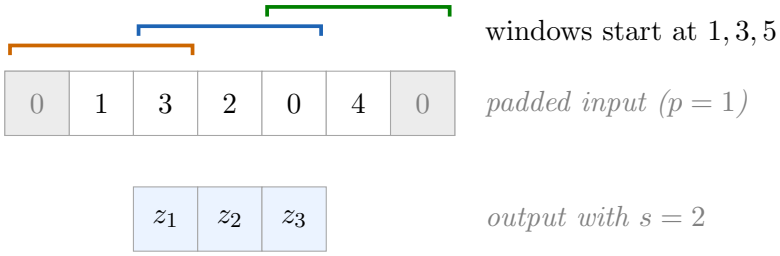


Figure 4: Stride $s = 2$: the kernel of width 3 jumps two positions at a time, giving three outputs instead of five.

Combining padding and stride gives the general output-size formula. In two dimensions it is applied to the height and the width separately.

For input size n, kernel size k, padding p and stride s:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

The floor $\lfloor \cdot \rfloor$ rounds down: a window that would stick out past the padded input is dropped.

Exercise 4 Output sizes

(a) Exercise 3 with stride $s = 2$: $n_{\text{out}} = \lfloor (5 + 2 - 3)/2 \rfloor + 1 = 3$, and $\mathbf{z} = (-3,\ 3,\ 0)$.

(b) Complete the table.

Input n	Kernel k	Padding p	Stride s	Output n_{out}
10	5	2	1	10
10	3	2	2	6
28	5	0	1	24
32	3	1	1	32
7	3	0	2	3
224	7	3	2	112

The last row is the first layer of ResNet (Section 4.4).

5 Channels

5.1 Multiple Input Channels

A colour image has $C_{\text{in}} = 3$ channels. A kernel for it is not a flat $k \times k$ grid but a *stack* of C_{in} grids, one per channel, so its shape is (C_{in}, k, k). Each slice is convolved with its own channel, and the results are **added** together with one bias. The kernel slides only across height and width, never across channels, so the output is a *single* feature map.

$$z_{i,j} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=1}^{k} \sum_{v=1}^{k} w_{c,u,v}\; x_{c,\, i+u-1,\, j+v-1} + b.$$

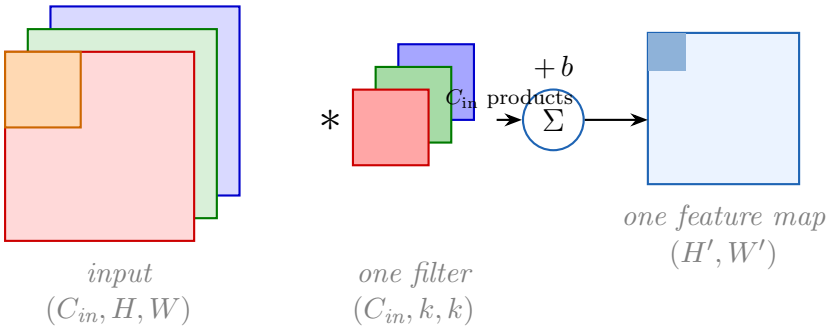


Figure 5: A filter has one $k \times k$ slice per input channel. Each slice meets its own channel; all products are added, plus one bias, to give one entry of a single output feature map.

Exercise 5 Two input channels

An input has two channels and a filter has two 2×2 slices, with bias $b = 1$ and no padding:

$$\mathbf{X}_1 = \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix}, \; \mathbf{X}_2 = \begin{pmatrix} 0 & 1 & 1 \\ 2 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \qquad \mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ 0 & 2 \end{pmatrix}, \; \mathbf{W}_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Compute each channel's contribution, then add them and the bias.

$$\underbrace{\begin{pmatrix} 3 & -2 \\ 6 & 3 \end{pmatrix}}_{\mathbf{X}_1 * \mathbf{W}_1} + \underbrace{\begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}}_{\mathbf{X}_2 * \mathbf{W}_2} + 1 = \underbrace{\begin{pmatrix} 7 & 0 \\ 8 & 6 \end{pmatrix}}_{\mathbf{Z}}$$

5.2 Multiple Filters

One filter detects one kind of feature. A convolutional layer uses C_{out} filters, each with its own weights and bias, and stacks the C_{out} feature maps into an output with C_{out} *channels*. That output is the input of the next layer, which is why its filters need C_{in} equal to the previous layer's C_{out}.

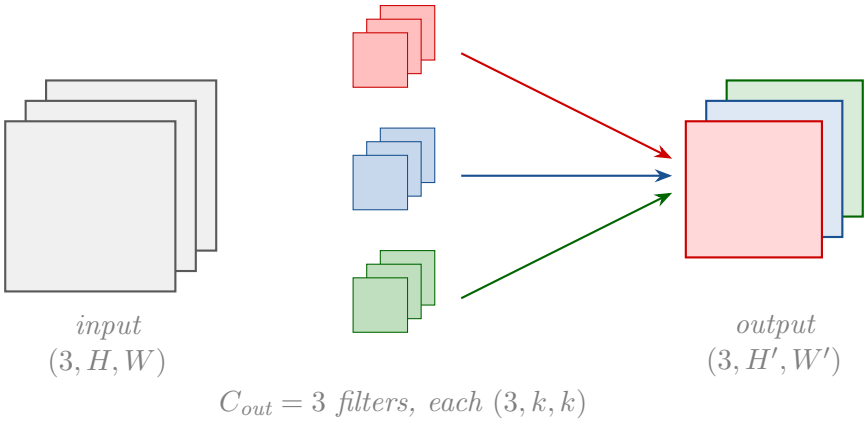


Figure 6: A layer with $C_{\text{out}} = 3$ filters produces an output with three channels: one feature map per filter.

A convolutional layer with C_{in} input channels, C_{out} filters and kernel size $k \times k$ has

- a weight tensor of shape $(C_{\text{out}}, C_{\text{in}}, k, k)$,
- a bias vector of shape (C_{out}),

$$\#\text{parameters} = (C_{\text{in}} \cdot k \cdot k + 1)\, C_{\text{out}}.$$

The count does *not* depend on the height or width of the image.

Compare this with the MLP formula $(n_{\ell-1} + 1)\, n_\ell$: one filter plays the role of one node, and its $C_{\text{in}} k^2$ weights are its inputs. The difference is that the same filter is reused at every position.

```
import torch
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=5)
x = torch.randn(64, 3, 32, 32)      # a batch of 64 colour images

print(conv.weight.shape)    # torch.Size([32, 3, 5, 5])
print(conv.bias.shape)      # torch.Size([32])
print(conv(x).shape)        # torch.Size([64, 32, 28, 28])
```

Exercise 6 Shapes and parameters

A batch of colour images of size 32×32 passes through two convolutional layers (stride 1).

Layer	Weight shape	Parameters	Output shape (one image)
Input	—	—	$(3, 32, 32)$
Conv: 32 filters, 5×5, valid	$(32, 3, 5, 5)$	2 432	$(32, 28, 28)$
Conv: 64 filters, 3×3, same	$(64, 32, 3, 3)$	18 496	$(64, 28, 28)$

A dense layer mapping the same input ($3 \cdot 32 \cdot 32$ values) to the same first output ($32 \cdot 28 \cdot 28$ values) would need

$$(3072 + 1) \times 25\,088 = 77\,095\,424 \text{ parameters},$$

about 32 000 times as many as the first convolution.

6 Convolution Is a Constrained Dense Layer

A convolution is linear in its input, so, like any linear map, it can be written as a matrix–vector product $\mathbf{z} = \mathbf{W}\mathbf{x}$, exactly as in a fully connected layer (Section 3.4). The matrix just has a special form.

Exercise 7 Write the convolution as a matrix

Take $\mathbf{x} = (x_1, \ldots, x_5)$ and $\mathbf{w} = (w_1, w_2, w_3)$, valid padding and no bias. Fill in the matrix so that $\mathbf{z} = \mathbf{W}\mathbf{x}$ gives $z_i = w_1 x_i + w_2 x_{i+1} + w_3 x_{i+2}$.

$$\begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix}$$

A general dense layer from 5 inputs to 3 outputs has 15 free weights. The convolution matrix has only 3, arranged in two kinds of constraint:

- **Zeros** outside a band: each output sees only its receptive field (sparse interactions).
- **Repeated entries** along each diagonal: every row is the row above shifted one place right (parameter sharing).

So a convolutional layer is a fully connected layer whose weights are forced to obey these constraints. Goodfellow et al. call this an *infinitely strong prior*: before seeing any data, we decide that the useful features are local and the same everywhere. For images this belief is correct, and that is why CNNs learn from far less data than MLPs.

7 Learning the Kernel: Backpropagation

The kernel weights are learned by gradient descent, so we need $\partial J/\partial w_u$. The matrix view tells us how: weight w_u appears in *every* row, once per output position. In Section 3.3 each weight affected the loss along one path. Here w_u affects the loss through every z_i, and the chain rule adds up all the paths.

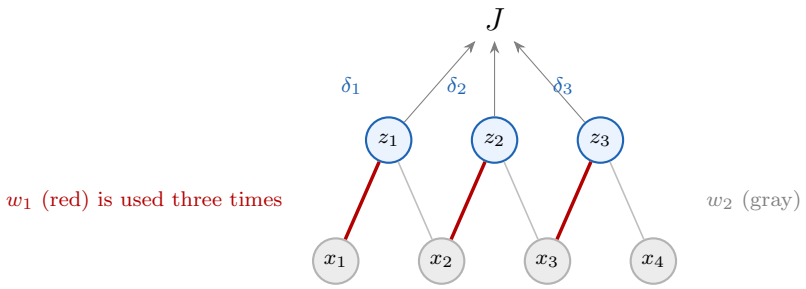


Figure 7: A kernel of width 2 on an input of length 4. Weight w_1 connects $x_1 \to z_1$, $x_2 \to z_2$ and $x_3 \to z_3$, so its gradient collects one term from each output.

Write $\delta_i = \partial J/\partial z_i$ for the gradient arriving at output i from later layers (as in Section 3.3). Since $z_i = \sum_u w_u x_{i+u-1} + b$, we have $\partial z_i/\partial w_u = x_{i+u-1}$ and $\partial z_i/\partial b = 1$. The chain rule, summed over all outputs, gives:

$$\frac{\partial J}{\partial w_u} = \sum_i \delta_i \, x_{i+u-1}, \qquad \frac{\partial J}{\partial b} = \sum_i \delta_i.$$

The gradient of a shared weight is the **sum** of the gradients from every position where it is used. Notice that $\partial J/\partial w_u$ is itself a convolution: slide $\boldsymbol{\delta}$ along $\mathbf{x}$.

Exercise 8 One gradient-descent step on a kernel

Let $\mathbf{x} = (1, 2, -1, 3)$, $\mathbf{w} = (w_1, w_2) = (1, -1)$ and $b = 0$. The desired output is $\mathbf{y} = (0, 2, -3)$, and the loss is $J = \frac{1}{2}\sum_i (z_i - y_i)^2$, so $\delta_i = z_i - y_i$.

(a) Forward pass: $\mathbf{z} = \big(-1,\ 3,\ -4\big)$ and $J = \frac{1}{2}(1 + 1 + 1) = 1.5$.

(b) Output gradients: $\boldsymbol{\delta} = \big(-1,\ 1,\ -1\big)$.

(c) Kernel and bias gradients:

$$\frac{\partial J}{\partial w_1} = \delta_1 x_1 + \delta_2 x_2 + \delta_3 x_3 = -1 + 2 + 1 = 2$$
$$\frac{\partial J}{\partial w_2} = \delta_1 x_2 + \delta_2 x_3 + \delta_3 x_4 = -2 - 1 - 3 = -6$$
$$\frac{\partial J}{\partial b} = \delta_1 + \delta_2 + \delta_3 = -1$$

(d) Update with learning rate $\eta = 0.1$: $w_1 = 0.8$, $w_2 = -0.4$, $b = 0.1$.

(e) New forward pass: $\mathbf{z} = \big(0.1,\ 2.1,\ -1.9\big)$ and $J = 0.615$. The loss went down.

PyTorch reproduces these numbers with automatic differentiation:

```
conv = nn.Conv1d(in_channels=1, out_channels=1, kernel_size=2)
with torch.no_grad():
    conv.weight[:] = torch.tensor([[[1., -1.]]])
    conv.bias.zero_()

x = torch.tensor([[[1., 2., -1., 3.]]])   # shape (N, C, length)
y = torch.tensor([[[0., 2., -3.]]])
J = 0.5 * ((conv(x) - y) ** 2).sum()
J.backward()
print(conv.weight.grad)    # tensor([[[ 2., -6.]]])
print(conv.bias.grad)      # tensor([-1.])
```

Gradient with respect to the input. To keep backpropagating to earlier layers we also need $\partial J/\partial x_j = \sum_i \delta_i\, w_{j-i+1}$, summed over the outputs whose window contains x_j. This turns out to be a *full* convolution of $\boldsymbol{\delta}$ with the *flipped* kernel: the flip we avoided earlier comes back in the backward pass. Frameworks compute it for us; the important idea is the one in the box above. Shared weights *add* their gradients.

8 Summary

Quantity	Formula
1-D convolution	$z_i = \sum_u w_u\, x_{i+u-1} + b$
2-D, many channels	$z_{i,j} = \sum_c \sum_u \sum_v w_{c,u,v}\, x_{c,\, i+u-1,\, j+v-1} + b$
Output size	$n_{\text{out}} = \lfloor (n + 2p - k)/s \rfloor + 1$
Same padding	$p = (k-1)/2$
Parameters of a layer	$(C_{\text{in}} k^2 + 1)\, C_{\text{out}}$
Kernel gradient	$\partial J/\partial w_u = \sum_i \delta_i\, x_{i+u-1}$

- A convolution slides a kernel over the input and takes a weighted sum at each position. Deep learning uses the unflipped version (cross-correlation).
- Padding controls whether the output shrinks (valid) or keeps its size (same). Stride s skips positions and shrinks the output about s-fold.
- A filter has one slice per input channel; a layer has C_{out} filters and produces C_{out} feature maps. Weights in PyTorch have shape $(C_{\text{out}}, C_{\text{in}}, k, k)$.
- A convolution is a dense layer with a banded, repeated weight matrix.
- The gradient of a shared weight is the sum over every position where it is used.

Coming up. Section 4.3 adds the remaining building blocks (activation, pooling and the classifier head) and assembles a complete CNN.